Máquinas Virtuales y Virtualización del SO

Sistemes Operatius II

Oliver Díaz

Grau d'Enginyeria Informàtica

¿How are you today?¿Ready for the test?



Test

wKon38_kl

Test Unidad 1: Introducción a la seguridad en SOs

S'obre el dijous, 1 d'octubre 2026, 15:15 **Es tanca el** dijous, 1 d'octubre 2026, 15:30

- Se realizará de manera **presencial** al aula
- 6 preguntas tipo test (4 posibles respuestas; solo 1 respuesta correcta)
- 15 min
- 1 único intento
- Respuesta correcta: 100%
- Respuesta incorrecta: -20%

15:15-15:30; Campus Virtual

Test

- **¿Os ha desbordado este primer test?**
- **Repasémoslo ahora de forma segura**



Pregunta 1

- **En un sistema con *Access Control Lists* (ACL), ¿qué ventaja se obtiene respecto al modelo clásico de permisos (`rwx`)?**

  a) Mayor velocidad de acceso

  b) Control más granular sobre usuarios específicos

  c) Eliminación de los permisos de grupo

  d) Posibilidad de ejecutar código en *kernel mode*

Pregunta 2

- **¿Cuál de los siguientes escenarios es un ejemplo típico de vulnerabilidad de tipo buffer *overflow*?**

  a) Escritura fuera de los límites de un array

  b) Llamada recursiva sin condición de parada

  c) Acceso concurrente a un fichero bloqueado

  d) Saturación de la red por tráfico SSH

Pregunta 3

- **¿Qué técnica intenta proteger la pila de ejecución insertando valores de control entre el búfer y la dirección de retorno?**

  a) ASLR

  b) DEP

  c) Canarios (pío-pío)

  d) PIE

Pregunta 4

- **¿Qué opción de compilación hace que un ejecutable se cargue en direcciones aleatorias cada vez que se ejecuta?**

  a) -no-pie

  b) -z execstack

  c) -pie

  d) -O2

Pregunta 5

- **Identifica la afirmación INCORRECTA sobre mecanismos de protección ante *buffer overflow*.**

a) ASLR dificulta predecir direcciones de memoria

b) DEP evita ejecutar código en zonas de datos

c) Los canarios detectan corrupción de la pila

d) PIE desactiva las comprobaciones de límites de los buffers

Pregunta 6

- **¿Cuál de las siguientes acciones reduce el riesgo de vulnerabilidades por *buffer overflow*?**

  a) Usar funciones como gets() o strcpy()

  b) No validar el tamaño de entrada del usuario

  c) Usar funciones como fgets() o strncpy()

  d) Desactivar ASLR para depurar programas

Noticias



Noticias



INSTITUCIONAL 30-09-2026

Sessions informatives sobre mobilitat internacional per al curs 2027-2028

La Facultat organitza dues sessions informatives sobre mobilitat internacional per al curs 2027-2028: una sobre els programes Erasmus+ i Global Faculty i una altra sobre Global Mobility UB.

Miércoles 7 de octubre a las 12:30 (aula B3)



INSTITUCIONAL 28-09-2026

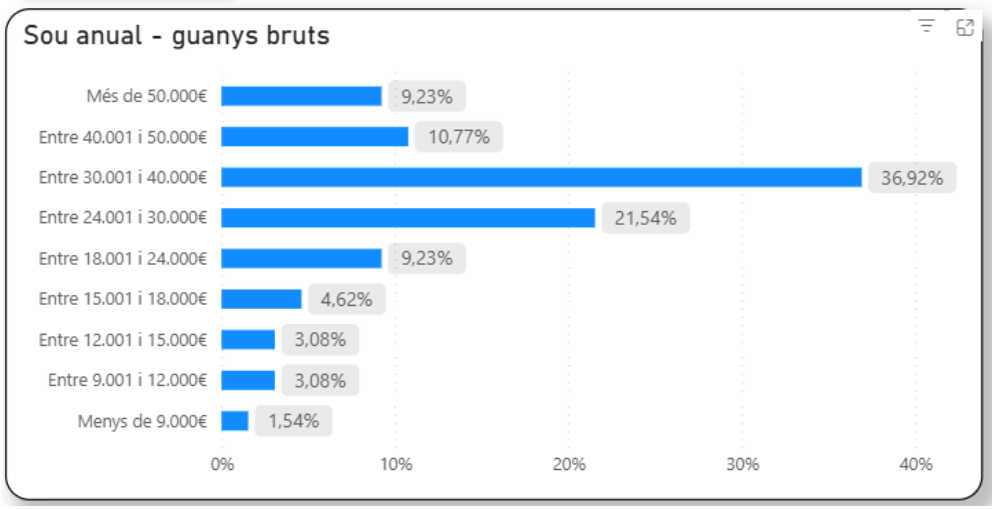
Teresa Ribera inaugura el curso académico en la Universidad de Barcelona

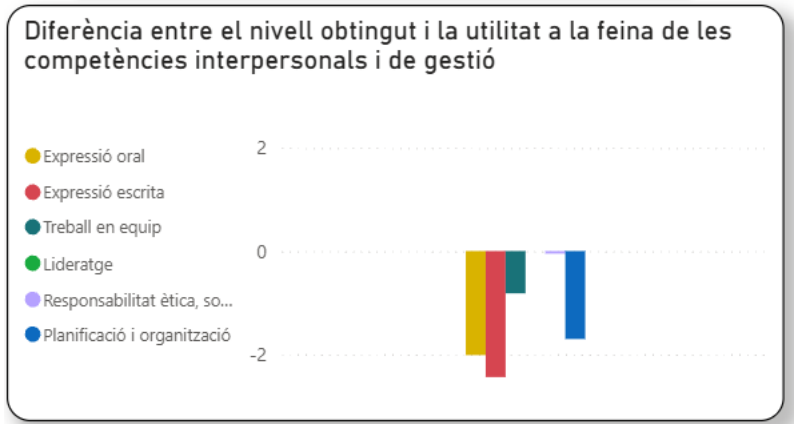
El acto, presentado por el comunicador Patrick Urbano, girará en torno a la sostenibilidad y la inteligencia artificial.

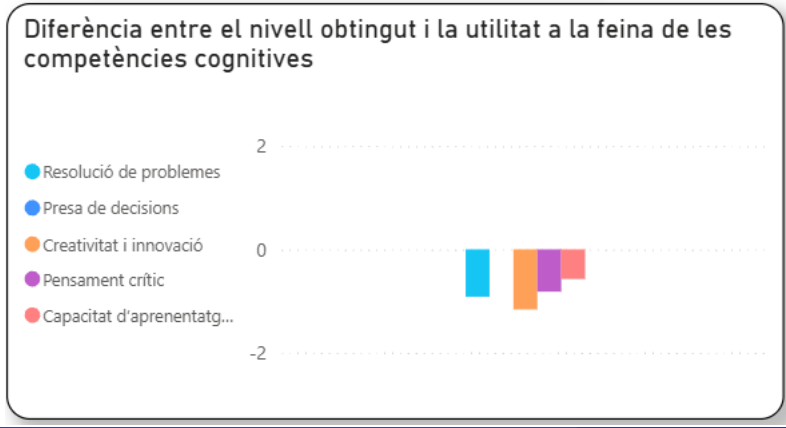
Lunes 5 de octubre a las 12:00 (Paraninfo)



- [Estudio de inserción laborar de AQU Catalunya]







Máquinas Virtuales

Sistemes Operatius II

Oliver Díaz

Grau d'Enginyeria Informàtica

Objetivos de la sesión de hoy

- Matizar (aún más si se puede) la definición de SO
- Saber que son las máquinas virtuales (MVs)
- Aprender los tipos de MVs que existen
- Conocer/reflexionar sobre sus usos y beneficios
- Investigar las soluciones empresariales existentes

La mayoria de nosotros utilizamos **Máquinas Virtuales** (VMs) en nuestro ordenador (SO1, SO2).

Pero en entornos empresariales, ¿son útiles?

- ¿Para qué una empresa utiliza las MVs?
- ¿Qué empresas las utilizan?
- ¿Cómo las utilizan?

¿Qué es la JVM?

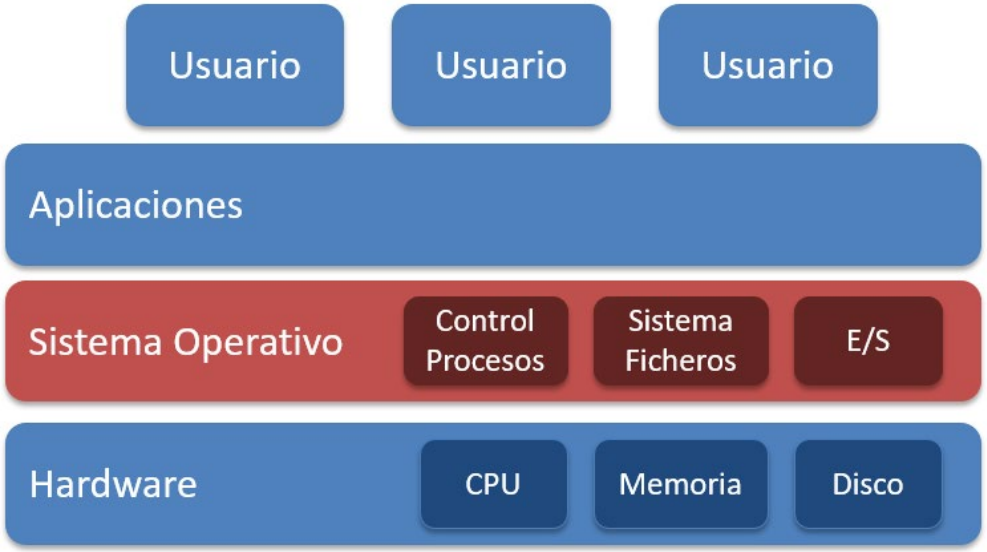


JVM
(Java Virtual Machine)

Filosofía “*write once, run anywhere*”

Abstrae el *hardware* subyacente y proporciona un entorno de ejecución virtualizado para aplicaciones Java (independiente del *hardware*)

Repasemos (una vez más) los SOs



Un SO se puede ver como una “máquina extendida”

- Oculta al usuario todos los **mecanismos complejos** que se realizan internamente (p.ej. acceso a los dispositivos)
- Ofrece al usuario una **MV** y una **interfície** (llamadas al sistema), facilitando así su uso

Repasemos (una vez más) los SOs

El SO permite la ejecución de **procesos** (instáncias de programas):

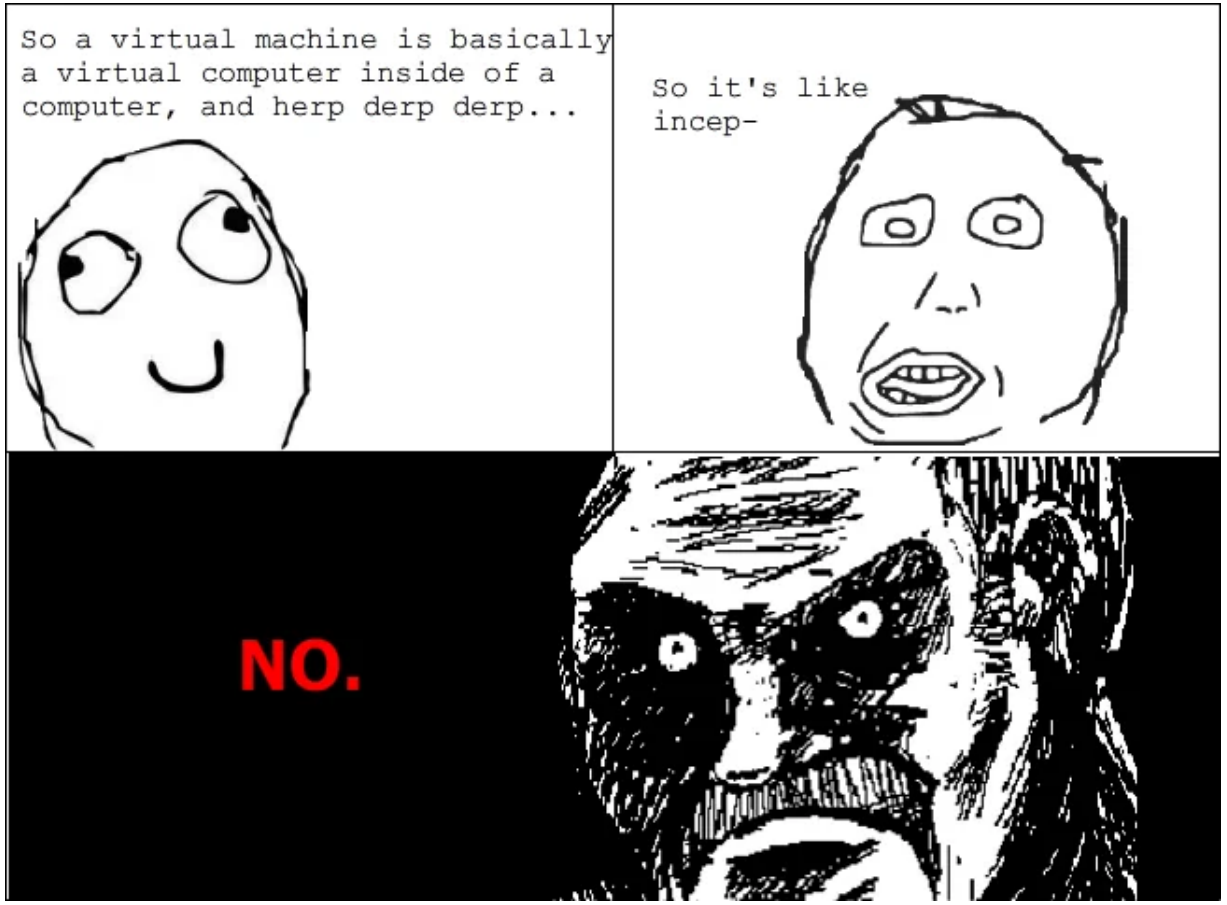
- Cada proceso tiene la **ilusión** de utilizar toda la máquina (100% recursos)
- Cada proceso puede **ejecutar instrucciones** en **modo usuario**. Además, puede **acceder a los dispositivos** de *I/O* a través del SO
- Un "*bug*" en una aplicación o "*malware*" no deberia afectar al resto de procesos (**aislamiento**)
- Un proceso no deberia consumir todos los **recursos** en detrimento del resto de procesos

El SO consigue todo eso **gestionando los recursos** disponibles, así como mediante el uso de técnicas de tiempo compartido (*time-sharing*) del *hardware*.

¿Entonces, son los SOs máquinas virtuales?

¿Son los SOs máquinas virtuales?

Primero veamos que es una **máquina virtual**:





https://www.reddit.com/r/ffffffffuuuuuuuuuuuu/comments/k0bjr/virtual_machine_rage/

¿Son los SOs máquinas virtuales?



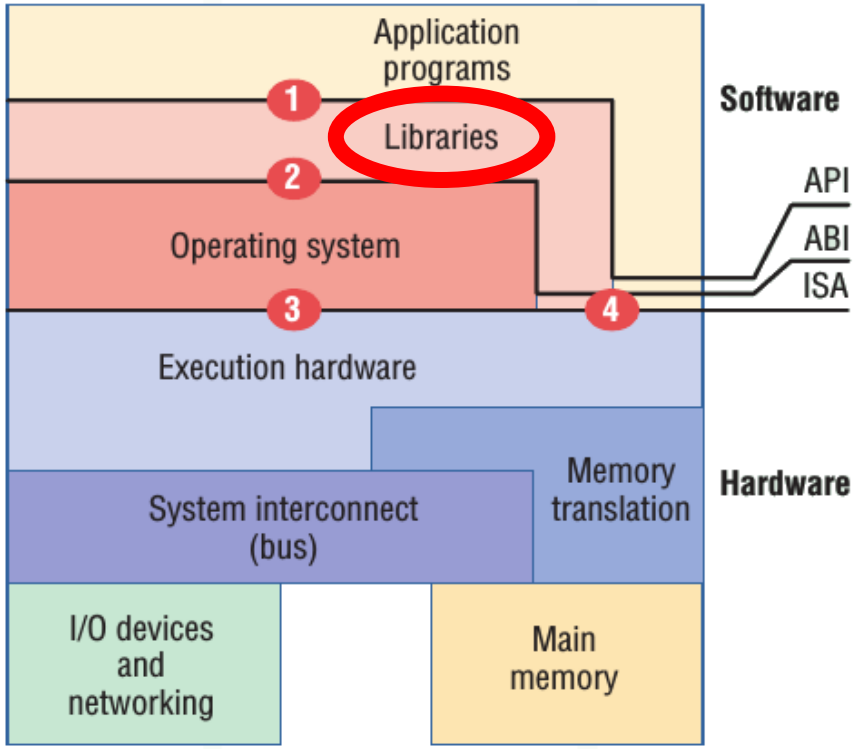
*"A Virtual Machine (VM) is a compute resource that **uses software instead of a physical computer** to run programs and deploy apps. One or more virtual **"guest" machines** run on a physical **"host" machine**. Each VM runs its own OS and functions separately from the other VMs, even when they are all running on the same host. This means that, for example, a virtual MacOS VM can run on a physical PC with Windows"*

https://www.vmware.com/topics/glossary/content/virtual-machine.html

¿Son los SOs máquinas virtuales?

De forma estricta, **un SO no es por definición una MV**. Entonces, ¿Qué es un SO?

- Software que tiene acceso directo al hardware y ofrece un **nivel de abstracción** con respecto a los procesos que se ejecutan en el SO.
- Esta diseñado para aprovechar la arquitectura del *hardware* para **garantizar la seguridad** a la hora de ejecutar **procesos**, dándo la ilusión que tiene recursos ilimitados (memoria infinita, 100% tiempo CPUs,…)

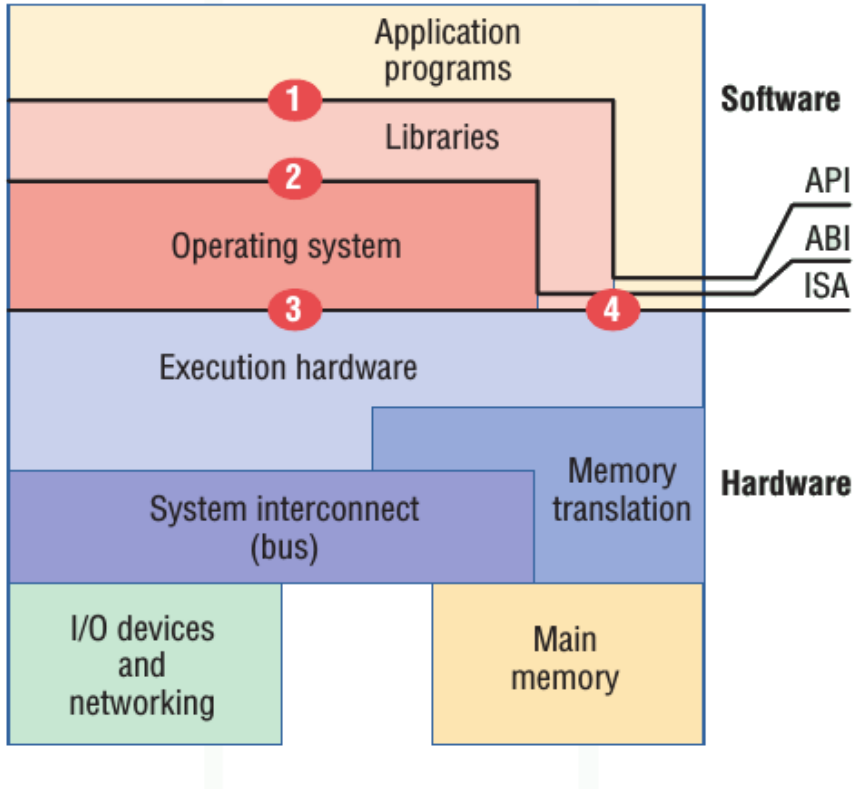


- **Librerias de usuario** facilitan el uso de herramientas que ofrece el SO (p.ej., **libc** → printf, fwrite...)

¿Son los SOs máquinas virtuales?

¿Què restricciones tiene?

- Esta diseñado para una determinada **arquitectura**
- Se ejecuta para un determinado **conjunto de instrucciones**
- Es el **único software** que controla el *hardware*



Limitamos entonces la flexibilidad del sistema

¿Son los SOs máquinas virtuales?



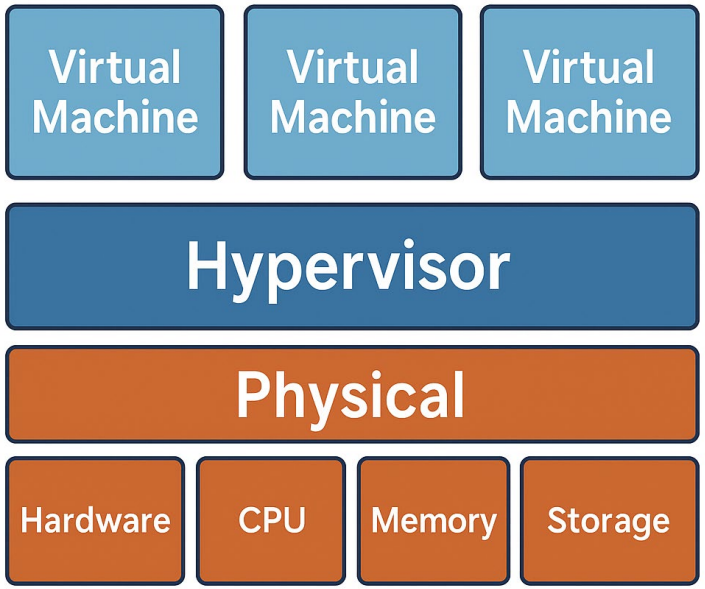
Actividad: ¿Virtualizar o no virtualizar?



- En **grupos de 3–4 personas** (5–7 min)
- **Escenario**: “Una empresa necesita desplegar 50 servidores lógicos para una nueva aplicación web. Tiene un presupuesto limitado, pero necesita seguridad, flexibilidad y alta disponibilidad.”
- **Preguntas**:
  - ¿Qué ventajas tendría utilizar máquinas virtuales?
  - ¿Qué inconvenientes o riesgos tendría frente a utilizar hardware físico dedicado?
  - ¿Compraríais 50 servidores físicos o un número menor de servidores potentes? ¿Por qué?
  - Bonus: Si un servidor físico ejecuta 10 VMs y falla, ¿qué ocurre? ¿Cómo lo solucionaríais?

La virtualización

- "Virtualización es el proceso de **abstraer los recursos físicos de un sistema** (CPU, memoria, almacenamiento, red) para crear **entornos virtuales independientes** que pueden ejecutarse de forma simultánea y aislada **sobre un mismo sistema físico**"



La virtualización

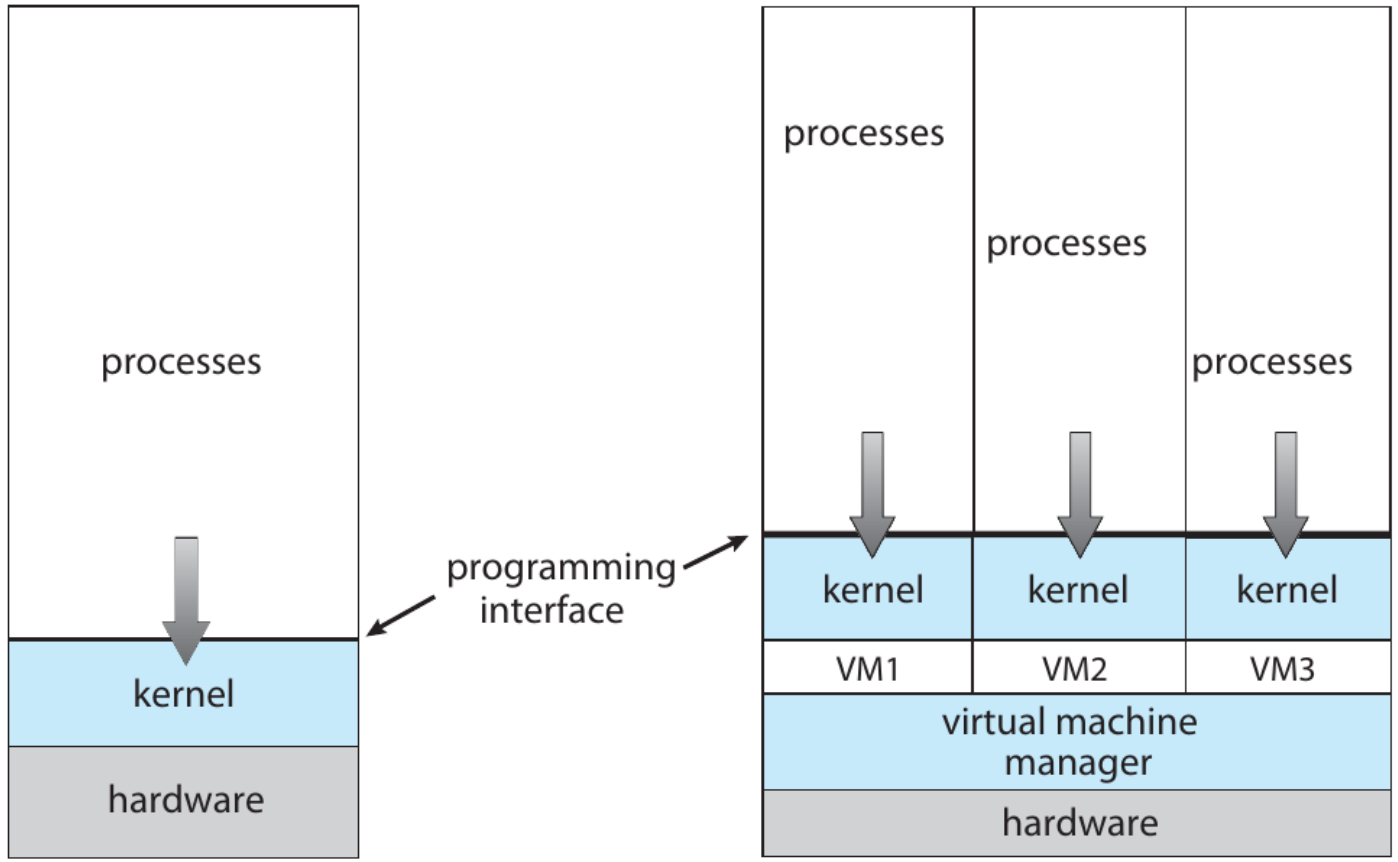
- La **virtualización** tiene como objetivo dar más flexibilidad a las restricciones del SO
- Al virtualizar un sistema (CPU, memória, dispositivos I/O, tarjeta de red), su interfície y recursos se **mapean** a la interficie y recursos del sistema real
- De esta forma, el sistema real se transforma de manera que parece ser un sistema virtual diferente[1]

[1] Una tarjeta gráfica o de red es puede mapear a una tarjeta gráfica o de red diferente en el sistema virtual

Las MVs

Las MVs más habituales involucran diversos componentes:

- Una Virtual Machine Manager (VMM), aka **hypervisor**
- Diversas MVs



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Las MVs

Las MVs más habituales involucran diversos components:



- **Host**: Corresponde con el sistema físico.
- **VMM**: Responsable de la **virtualización/emulación …**
- …y tiene la funcionalidad de un ordenador físico (puede ofrecer un *hardware* diferente al que existe físicamente).
- La VMM puede ejecutar diversas MVs, y proveer al proceso **guest** de una copia virtual del host.
- Típicamente el **proceso guest** es un SO.

(Virtualización vs Emulación)

- **Virtualización**: El *hardware* físico y virtual comparten la misma arquitectura.
  - VirtualBox ejecutando Ubuntu en un PC Intel/AMD

- **Emulación**: El *hardware* virtual simula una arquitectura distinta al *hardware* físico (permite compatibilidad a costa de rendimiento).
  - QEMU emulando una CPU ARM sobre un PC x86
  - Consolas antiguas (PlayStation) emuladas en un PC moderno

Las MVs

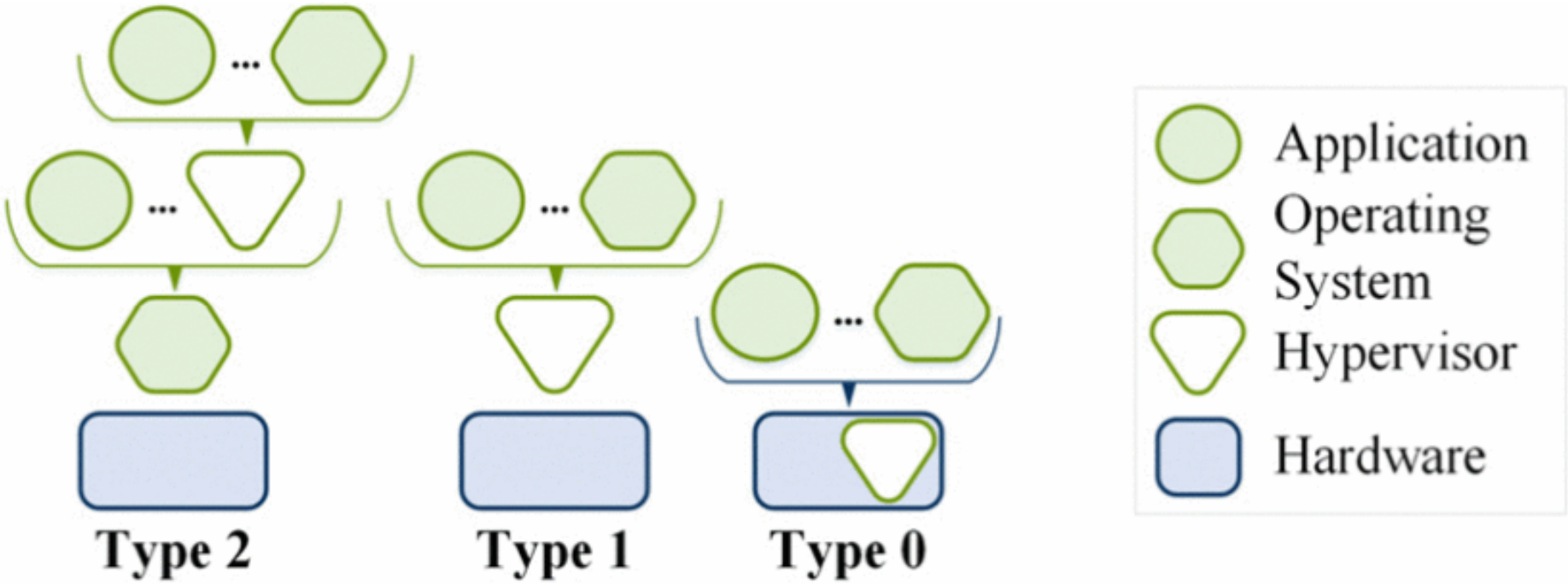
- Cada SO se diseña para tener el control total del *hardware*.
- Sin embargo, en una MV el VMM:
  - Hace creer a cada **SO guest** que tiene el control total del sistema (aunque se ejecuta en **modo usuario**)
  - Los procesadores más modernos incluyen **instrucciones máquina** que facilitan esta tarea. A las arquitecturas x86 se conocer como **instrucciones VTX**.
- Las instrucciones VTX permiten realizar el "**trap-and-emule**":



  - Un SO guest realiza una operación no permitida (p.ej., escritura de datos a disco)
  - El procesador captura ("*trap*") esta instrucción
  - El VMM ejecuta lo que quiere hacer el SO guest. Gracias al mapeado realizado por la MV, se ejecuta lo que el SO guest queria hacer

Tipos de MVs

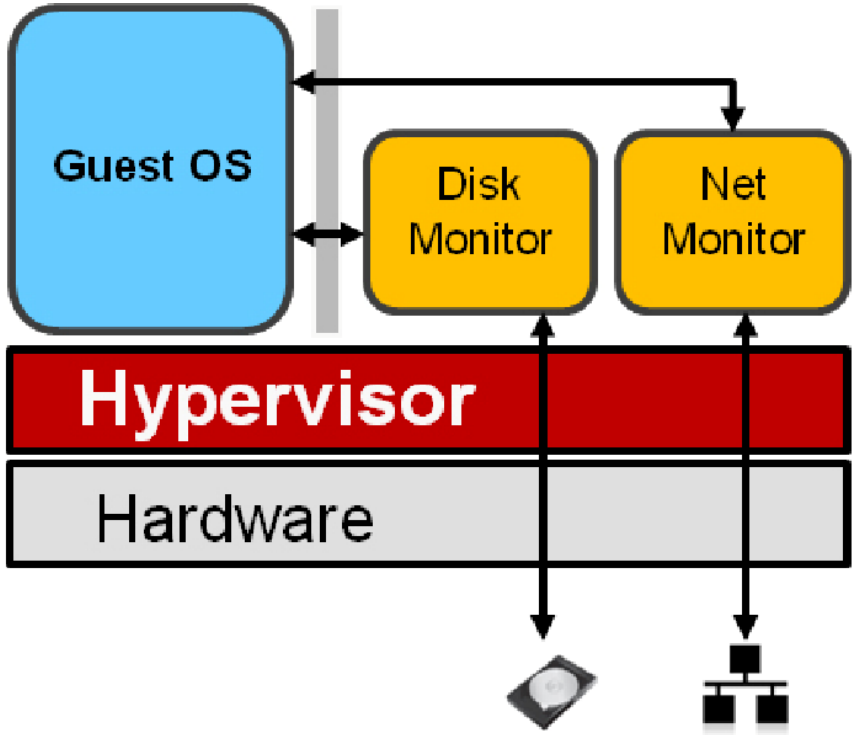
- Type 0 hypervisors
- Type 1 hypervisors
- Type 2 hypervisors



Janßen, Benedikt, et al. "Towards a type 0 hypervisor for dynamic reconfigurable systems." 2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig). IEEE, 2017.

Type 0 hypervisor

- No es común.
- Soluciones basadas en hardware que brindan soporte para la creación de máquinas virtuales y gestión vía ***firmware*** .
- Construido con los componentes de ***software*** **mínimos** necesarios para virtualizar completamente los SOs guests y controlar el flujo de información entre los SOs guests.



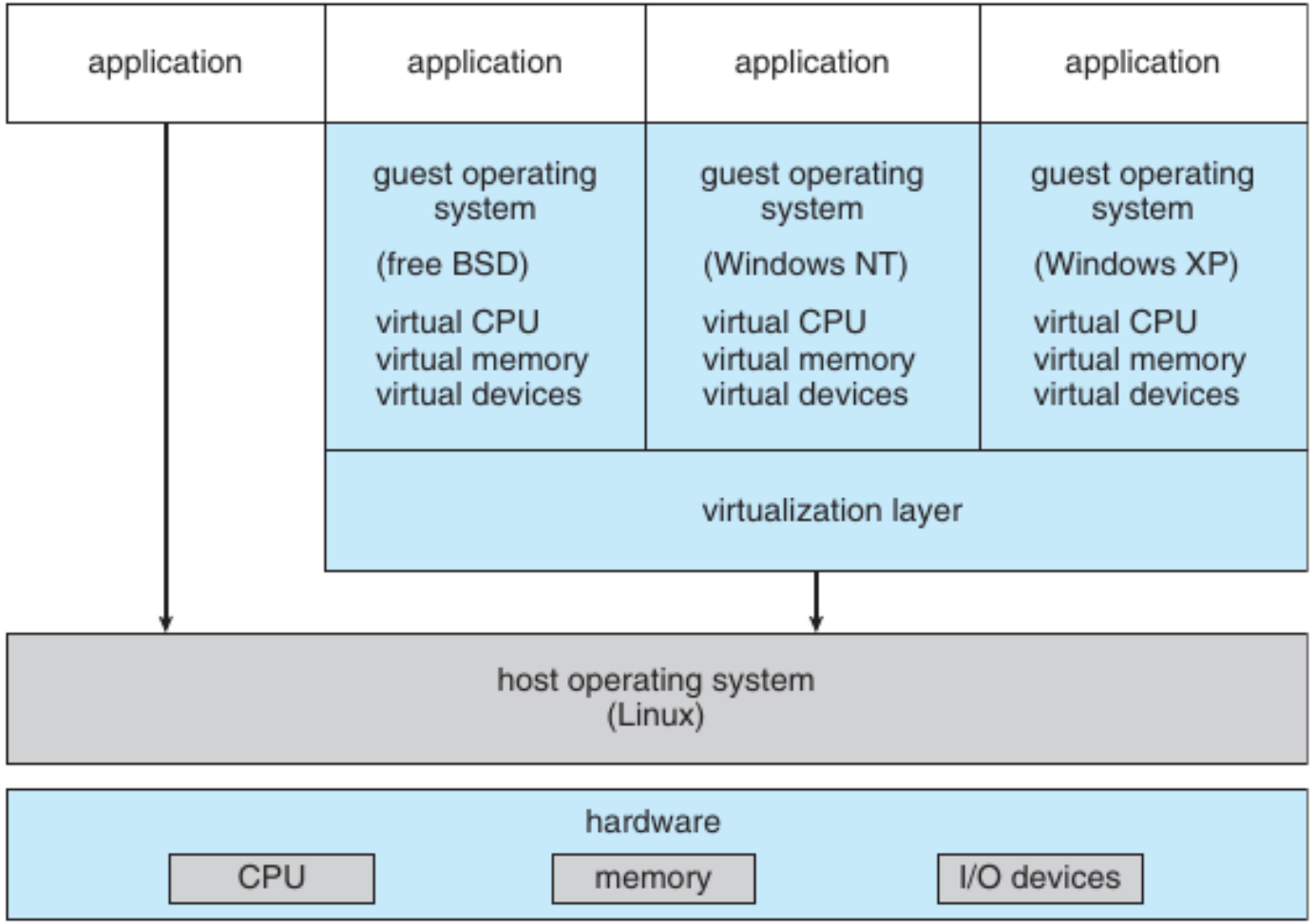
Keegan, Will, and Arun Subbarao. "Type Zero Hypervisor– the New Frontier in Embedded Virtualization."

Type 1 hypervisor

- Se encuentra en los grandes servidores.
- La VMM se instala al servidor. Al arrancarlo, éste se ejecuta directamente sobre el *hardware* y **provee de servicios tradicionales** de SOs como planificación o gestion de memoria.
- La VMM actúa como un SO que **permite ejecutar y gestionar otros SOs**.
  - E.g. VMWare ESXi: "*The virtualization software resides on the "bare metal" or the hard disk of the hardware, where the OS is usually installed*."
- Una única máquina puede ejecutar **multiples SOs simulatáneamente**, cada una con su propia MV. Cada SO cree tener el control total sobre la máquina (virtual).
- La virtualización es una técnica para proveer a cada SO guest de hardware (virtual).

Type 2 hypervisor

- Se ejecutan en un **SO convencional**
- El SO guest se ejecuta como si fuera un **proceso** del host.



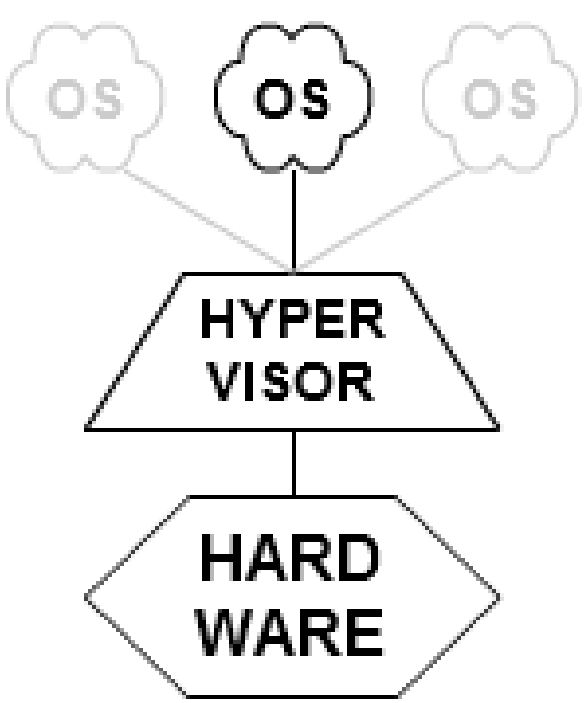
Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Type 2 hypervisor

- **WVMare Workstation**
  - Tiene su capa de virtualización que abstrae el *hardware* en MVs independientes que permiten ejecutar SO.
  - Cada MV tiene su propia CPU virtual, memoria, discos, red, etc.
  - El disco de una MV no es más que un archivo en el host. Al copiar este archivo podemos crear/clonar un *guest* idéntico al original

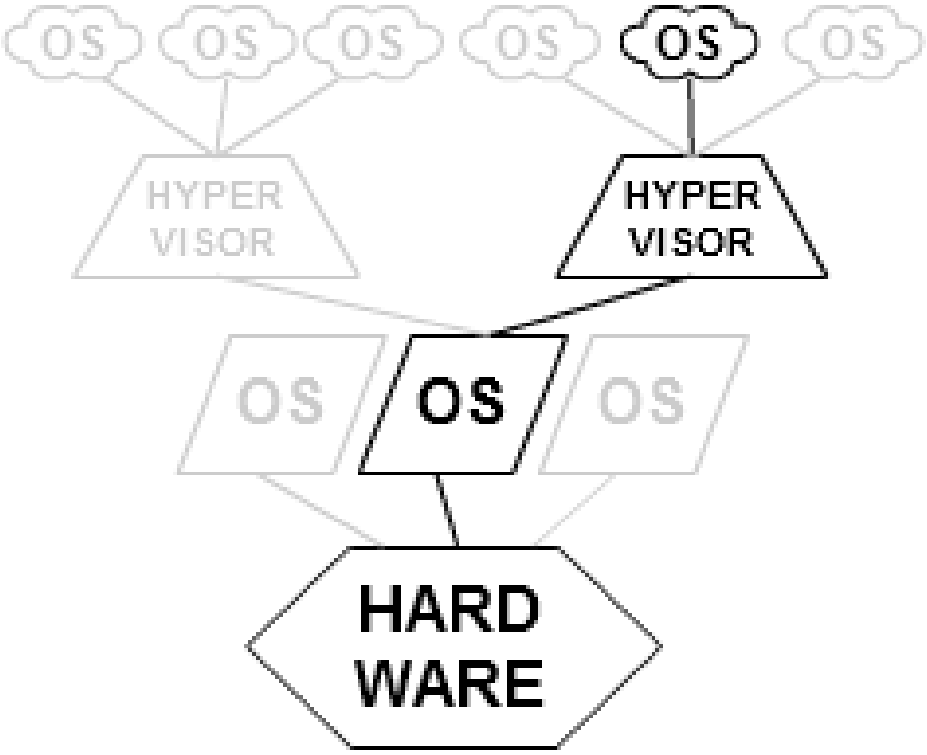


Type 1 y 2 hypervisor



TYPE 1

native
(bare metal)

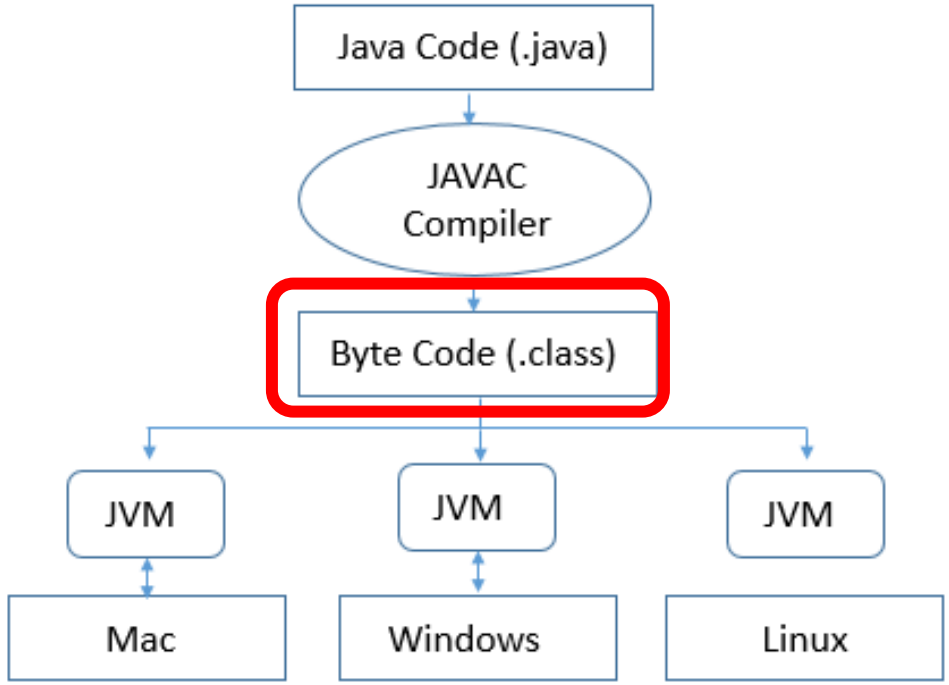


TYPE 2

hosted

Tipos de MVs (según su funcionalidad)

- Las MVs se pueden clasificar también segun su funcionalidad
  - **MVs de sistema**: Provee un sustituto de una MV y permite ejecutar un SO. Incluye los *type 1* y *type 2 hypervisors*.
  - **MVs de proceso**: Permiten ejecutar un proceso en un entorno independiente de la plataforma. Las VMM no virtualizan el *hardware* real sino un sistema virtual optimizado.
    - E.g. Java → bytecodes (código máquina abstracto)



¿Os acordais de Valgrind?

- **Valgrind** (/ˈvælgrɪnd/) es una herramienta de programación para la depuración de memoria, la detección de fugas de memoria y la creación de perfiles.



https://en.wikipedia.org/wiki/Valgrind

¿Os acordais de Valgrind?

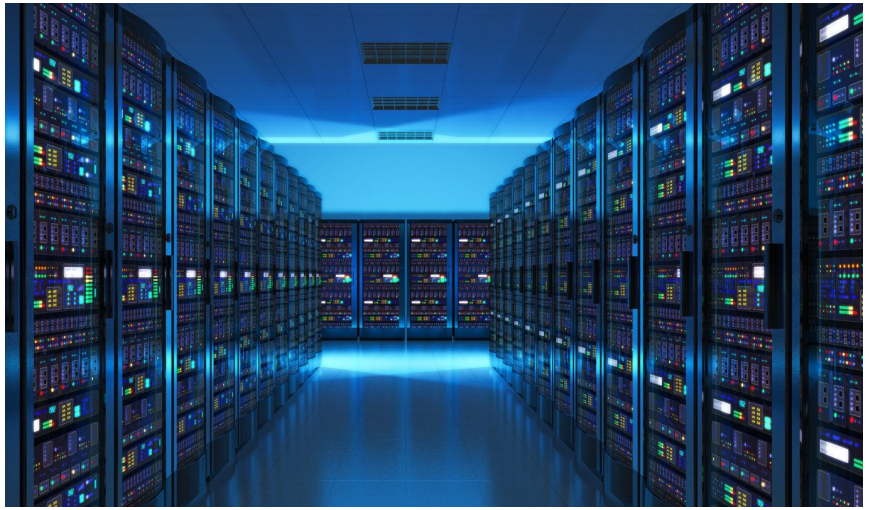
*"Valgrind is in essence a **virtual machine** using just-in-time (JIT) compilation techniques, including dynamic recompilation. **Nothing from the original program ever gets run directly on the host processor**. Instead, Valgrind first **translates the program into a temporary, simpler form called Intermediate Representation** (IR), which is a processor-neutral, SSA-based form. After the conversion, a tool (see below) is free to do whatever transformations it would like on the IR, before Valgrind translates the IR back into machine code and lets the host processor run it. **Valgrind recompiles binary code to run on host and target (or simulated) CPUs of the same architecture**."*

https://en.wikipedia.org/wiki/Valgrind

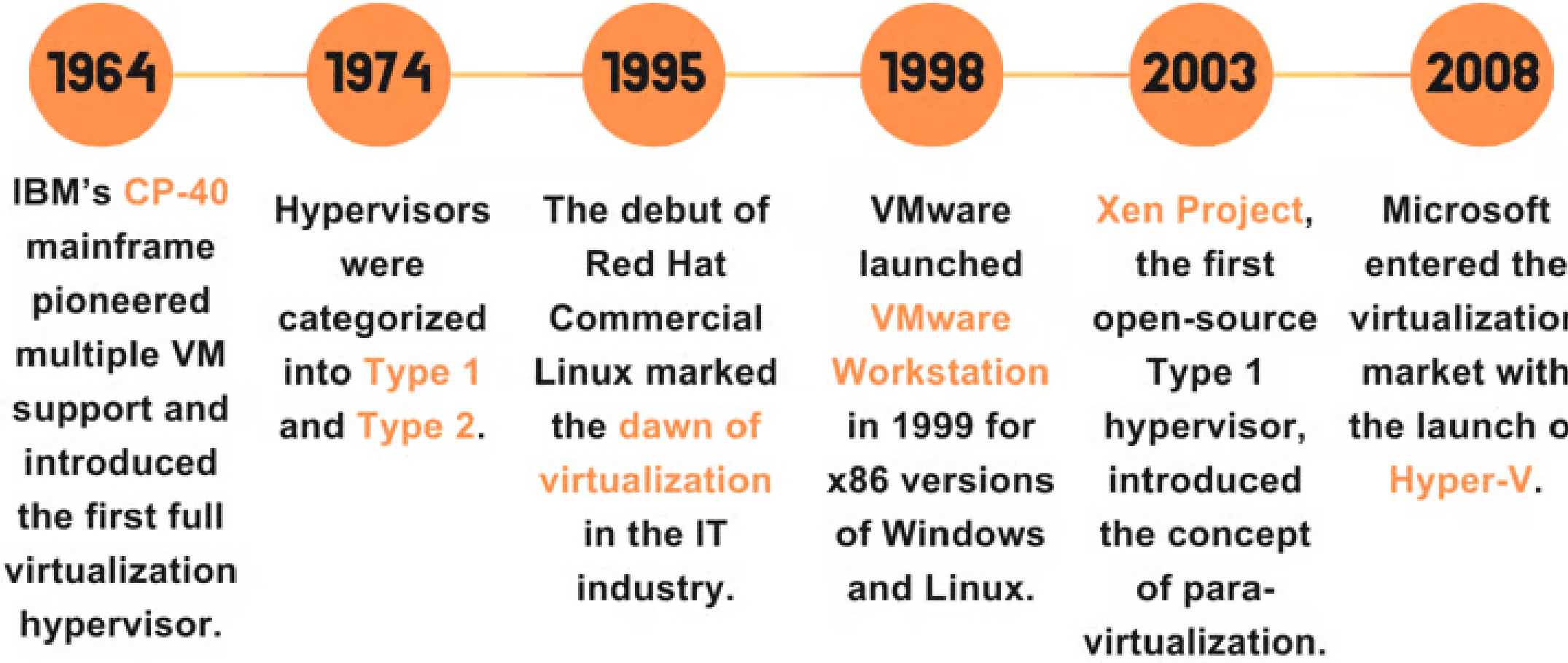
Historia de las MVs

- Aparecen en 1972 (a los *mainframes* de IBM) y utilizaban IBM VM, el cual proveía de MVs necesarias (IBM CP-40; IBM CP67) .
- Con la aparición de los ordenadores personales (abaratamiento de los PCs), el interés por las MVs disminuyó.
- Hoy en día las MVs vuelven a tener gran popularidad, sobre todo en los servidores/granjas de servidores.



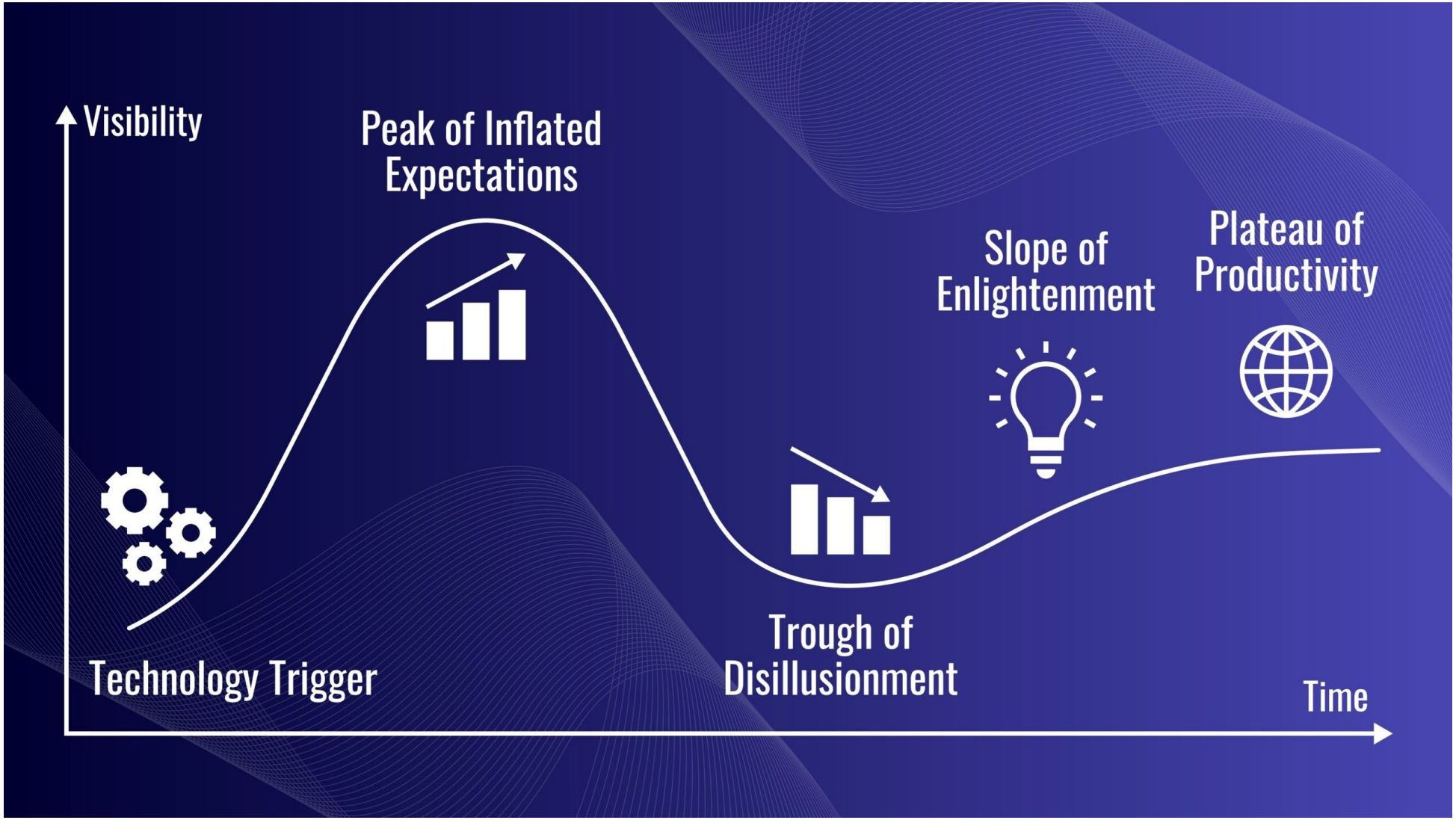


Historia de las MVs

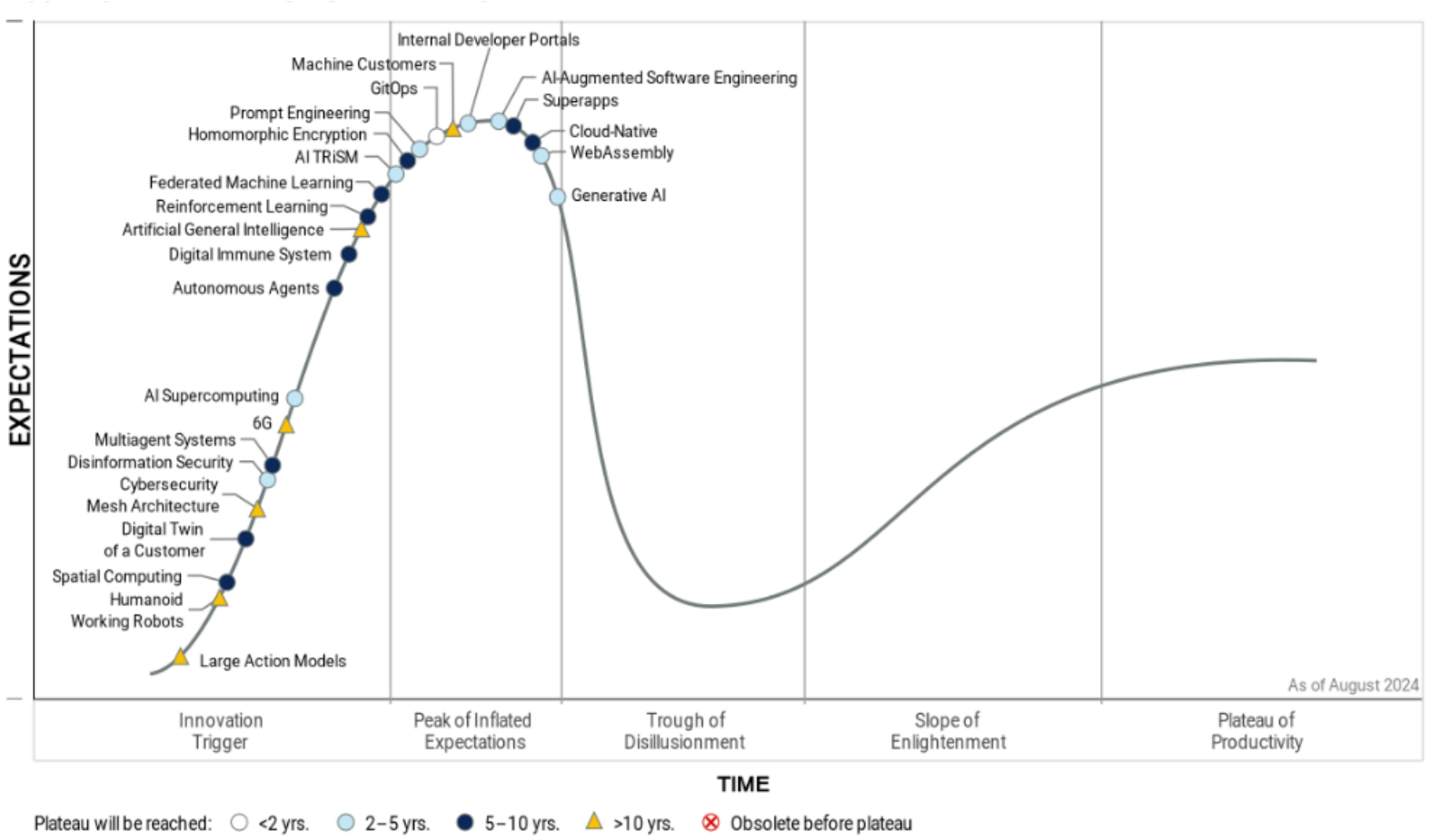


https://www.hostingadvice.com/how-to/what-is-a-virtual-machine/

Hype Cycle for Emerging Technologies, 2024



Hype Cycle for Emerging Technologies, 2024



Gartner

https://www.gartner.com/en/newsroom/press-releases/2024-08-21-gartner-2024-hype-cycle-for-emerging-technologies-highlights-developer-productivity-total-experience-ai-and-security

Beneficios y características de las MVs

- **Virtualización de escritorios**: una organización puede proporcionar escritorios virtuales (como la Facultad durante la pandemia) con MVs de la propia organización o externos.
- **Virtualización de servidores**: una máquina física puede gestionar múltiples servidores, cada uno con su propia MV y SO. Si un servidor falla, los otro no quedan afectados.
- **Recursos**: una organización que necesita (grandes) recursos computacionales puede subcontratar los servicios de una empresa externa especializada en gestionar infraestructuras asociadas a MVs.

Beneficios y características de las MVs

- Las MV están prácticamente aisladas entre si. Un usuario puede tener una **MV con recursos restringidos** (menos CPU, menos RAM) de la que realmente tiene disponible (máquina real).
- La virtualización permite **parar** una MV (con su SO), hacer una **copia** (clon), **moverla** a otra MV, **guardarla**, etc.
- La virtualización facilita el **desarrollo y testeo** de nuevos SO, así como el desarrollo y testeo de aplicaciones en diferentes SOs al mismo tiempo.

¿Quién utiliza MVs?

Empresas (grandes, medianas y pequeñas) pueden externalizar servicios para utilizar MVs. Las empresas utilizan estos servicios ya que ofrecen **escalabilidad**.

- **Google Cloud Platform**: Spotify, HSBC, Facebook, Domino, Sony Music, Ubisoft.
- **Amazon Web Services**: Siemens, Apple, Adobe, Pfizer, Nasa, Netflix.
- **IBM Cloud**: Koopman Logistics, Crédit Mutuel, Osram AH, RS Components.
- **Microsoft Azure**: Coca-Cola, Accenture, Adobe, Citrix, Symantec Corporation.



En resumen

La **virtualización** provee de duplicidad del *hardware* subjacente

- A las **MVs de sistema**, en cada VM se puede ejecutar un SO diferente y cada VM pude ofrecer un hardware diferente a cada SO.
- Dada su popularidad cada vez mayor, los diseñadores de CPUs añaden cada vez más características para facilitar el **soporte a la virtualización**.
- Esta arquitectura de virtualización tiene asociada una complejidad alta. Para determinadas aplicaciones es más util **virtualización a nivel de SO** (lo veremos la semana que viene)

Actividades participativas

Glosario Sistemas Operativos

Pàgina: **1** 2 (Següent)
TOTES

A

ABI - Application Binary Interface

Interfaz que define cómo los programas compilados interactúan con el sistema operativo, el hardware y otras bibliotecas. Establece aspectos como convenciones de llamada, paso de parámetros y representación de datos.

Dicho de otra manera: Una ABI es muy similar a una API.

Puedes pensar en ella como la **versión compilada de una API** (o como una API a nivel de lenguaje máquina). Cuando escribes código fuente, accedes a la biblioteca mediante una **API**. Una vez que el código se compila, tu aplicación accede a los datos binarios de la biblioteca mediante la **ABI**. La ABI define las **estructuras y los métodos** que utilizará tu aplicación compilada para acceder a la biblioteca externa (igual que hacía la API), pero a un nivel más bajo. Tu **API** define el orden en el que pasas los argumentos a una función. Tu **ABI** define los mecanismos de **cómo** se pasan esos argumentos (registros, pila, etc.).

Tu **API** define qué funciones forman parte de tu biblioteca. Tu **ABI** define cómo se almacena tu código dentro del archivo de la biblioteca, de manera que cualquier programa que utilice tu biblioteca pueda localizar la función deseada y ejecutarla.

Para más información: https://stackoverflow.com/questions/2171177/what-is-an-application-binary-interface-abi

Paraula o paraules clau: Application Binary Interface

ACL (Access Control Lists)

Definició:

Mecanisme de seguretat en sistemes de fitxers de Linux que amplia el model de permisos tradicional (usuari, grup, altres). Les ACLs permeten assignar permisos de lectura, escriptura o execució de forma granular i específica a usuaris o grups concrets que no són el propietari ni el grup principal del fitxer o directori. A més, les ACLs per defecte d'un directori permeten que els nous objectes creats a dins heretin automàticament aquesta estructura de permisos.

Exemple:

Suposem que tenim el fitxer `/tmp/SO2/acl1.txt` la propietat del qual pertany a `oslabadmin` i al grup `grupo1`. Si volem que un usuari extern com `oslab` tingui permisos de lectura i escriptura (`rw-`), i que el grup `grupo2` tingui només permisos de lectura (`r--`), podem configurar la seva ACL mitjançant la comanda

```
setfacl -m u:oslab:rw-,g:grupo2:r-- /tmp/SO2/acl1.txt
```

En executar `getfacl /tmp/SO2/acl1.txt`, es pot comprovar com `oslab` manté accés personalitzat sense necessitat de canviar el propietari del fitxer ni de fer-lo membre del grup principal.

Paraula o paraules clau: acl

Wiki de SO2

Foro de la asignatura

De la teoria a la cabina de vol: Buffer Overflows a VxWorks (URGENT/11) i la seguretat en sistemes de temps real (RTOS)
per VERGES SANTIAGO MARC - dimecres, 30 de setembre 2026, 19:26

Hola a tothom!

Després de veure les primeres diapositives de la introducció a l'assignatura de SO1, em va quedar molta curiositat per saber més sobre els sistemes operatius que s'utilitzen en entorns crítics on la seguretat el temps de resposta són absolutament vitals, com p.e. l'aviació, i en particular l'exemple que allà s'exposava: VxWorks. Com que a més m'apassiona tot el món de l'aviació, m'he posat a investigar com funcionen els **Sistemes Operatius en Temps Real (RTOS)** que porten els avions moderns (com el Boeing 787 o l'Airbus A350) i quin tipus de vulnerabilitats poden patir.

Investigant sobre un dels SOs més utilitzats en la indústria aeronàutica i en la NASA, **VxWorks** (de Wind River Systems), vaig trobar un cas d'estudi real que enllaça directament amb els conceptes que estem treballant a la Pràctica 1: la col·lecció de vulnerabilitats coneguda com a URGENT/11.

1. El cas d'estudi: URGENT/11 a VxWorks

VxWorks és un RTOS (Real-Time Operating System) dissenyat per a garantir el determinisme temporal. Tot i la seva reputació de robustesa, investigadors de seguretat van descobrir 11 vulnerabilitats des del dia zero a la seva pila de xarxa TCP/IP, de les quals 6 permetien Execució Remota de Codi (RCE) sense cap tipus d'autenticació o interacció de l'usuari.

2. Connexió tècnica amb la Pràctica 1

Analitzant el detall d'aquesta fallada, s'hi veuen reflectits exactament els punts teòrics del nostre laboratori:

- **Stack Overflow vs. Heap Overflow:** Dins d'URGENT/11 trobem tant desbordaments de memòria a la pila (com el **CVE-2019-12256**, en analitzar opcions d'IPv4) com desbordaments a la memòria dinàmica o *heap* (com el **CVE-2019-12257**, en la lectura de missatges DHCP).
- **Absència de Mitigacions:** Als apunts hem après que mecanismes com **ASLR, NX/DEP o Canaris de la pila** ajuden a mitigar o dificultar l'explotació d'aquests errors. L'informe d'URGENT/11 destaca que, tot i que VxWorks incloïa la possibilitat d'activar aquestes proteccions, els fabricants de maquinari gairebé mai les tenien activades, deixant els dispositius exposats.

Pregunta per a la discussió:

A la pràctica hem comprovat com de fàcil és cometre un error de gestió de memòria en llenguatges com C. En entorns tan crítics com l'aviació o la medicina, on els equips funcionen durant dècades sense poder actualitzar-se fàcilment i veient que en sistemes reals com VxWorks aquestes proteccions sovint venien desactivades per defecte:

Creieu que els compiladors i SOs haurien d'obligar a tenir activats els canaris i l'ASLR per defecte a tot arreu, o és comprensible que en sistemes encastats es desactivin per estalviar recursos de CPU?

Us llegeixo i m'agradaria saber què en penseu!

Enllaç a la notícia: https://thehackernews.com/2019/07/vxworks-rtos-vulnerability.html.

Entrega P1 mañana

Entrevista P1 semana próxima

Actividades participativas

